

Lab 2 - Estimation of Motion Variables

Course notes – 2025/26 labs

Contents

1 Lab handout - objectives and tasks	1
2 Reference paper - Partial attitude and rate gyro bias estimation (Batista, Silvestre, Oliveira 2011)	3
3 Devkit guide/scripts - NAV-specific additions vs Lab 1's devkit	5
4 Simulink models present	7
5 Connections to lecture material	7
6 Open questions / unclear points	8

Source files:

- 25_26_UAVs_Lab2.pdf (6 pages) - lab handout, "Estimation of Motion Variables of the Parrot AR.Drone," Técnico Lisboa, MSc Aerospace Engineering, 2025/2026 First Semester, October 2025.
- Partial attitude and rate gyro bias estimation observability analysis filter design and performance evaluation.pdf (9 pages) - P. Batista, C. Silvestre, P. Oliveira, *International Journal of Control*, vol. 84, no. 5, pp. 895-903, May 2011.
- DevKit_Lab2/ARDroneSimulinkDevKit_NAV/ - all .m scripts (start_here_NAV.m, lib/decode_32bit.m, lib/getWaypoints.m, lib/setupARModel.m, Replay/setupReplay.m, simulation/setupBaseModel.m, simulation/setupHoverSim.m, simulation/setupWPTrackingSim.m, wifiControl/setupHover.m, wifiControl/setupWPTracking.m).

Handout is in English. All pages/files processed in full.

1 Lab handout - objectives and tasks

Objective: design, implement, and analyze several estimation solutions for the **roll and pitch angles** of the Parrot AR.Drone quadrotor, in simulation and experimentally.

Deliverable: single-column PDF report, max 15 pages, plus the .m scripts, submitted via Fenix using the official cover page (25_26_UAVs_Lab_report_cover_page.docx, already in this project). Questions are tagged **(T)** theoretical (to be

solved before the lab session) or **(L)** laboratory (simulation/experiment, done during the session).

Section 2 - Setup and experiments. New devkit features vs. Lab 1:

- Entry script is now `start_here_NAV.m` (adds a 3rd top-level option: replay stored data).
- Wi-Fi control modes now log raw navigation data into a workspace variable `navdata` at **200 Hz**.
- The `navdata-decoding` block is an "extended version" exposing **accelerometer and rate-gyro** samples (Lab 1's devkit apparently only exposed velocity/attitude estimates, not raw IMU data).

Five data-collection experiments define the whole lab's empirical basis:

- **Experiment A:** sensors at rest, motors off, Hover mode, no take-off, 30 s - baseline/quiescent noise characterization.
- **Experiment B:** on the ground, motors spinning, hull held down to prevent lift-off, take-off command sent, 30 s, then land - isolates motor-vibration-induced noise.
- **Experiment C:** real hover flight, take-off → 30 s hold → land - full operational noise (motor vibration + aerodynamic disturbance + control activity).
- **Experiment D:** step response in pitch reference, by editing `ARDroneHover.slx` (same style of step test as in Lab 1) - used to see filter behavior under a real, non-stationary attitude change.
- **Experiment E:** short waypoint mission - edit `ARDroneHover.slx` to give a 2-waypoint reference in `y`: origin → $[0, 0, 0.75]$ → $[0, 1, 0.75]$ → back to $[0, 0, 0.75]$ → land, 5 s dwell between reference changes. Used to see estimator behavior across a realistic short mission.

Section 3 - Modeling and characterization of sensors (all "(L)"):

- 3.1-3.3: compute mean and covariance of accelerometer + rate-gyro readings for Experiments A, B, C respectively, and discuss *why* uncertainty grows from A→B→C (new disturbance sources: motor vibration in B, aerodynamic/control-induced disturbance in C).
- 3.4: compute "raw" pitch/roll from the accelerometer alone (the inclinometer formulas $\hat{\phi}_{accel} = \text{atan2}(\dots)$, $\hat{\theta}_{accel} = \sin^{-1}(\dots)$ documented in [../../../../lectures-22-23/notes/03-sensors-estimation.pdf](#)), to serve as the *baseline* against which every filter built later in the lab is compared.

Section 4 - Kalman filters (pitch first, then roll by symmetry in 4.7):

- 4.1: design/implement a **steady-state Kalman filter** for pitch using the pitch inclinometer measurement θ_m and rate-gyro measurement w_{ym} (note: MATLAB's `kalman()` or `estim()` commands are explicitly suggested - i.e. this is the LTI/steady-state KF machinery from [../../../../lectures-22-23/notes/02-control.pdf](#) Ch4, not a hand-rolled recursive filter).
- 4.2: derive, for the constant (steady-state) Kalman gains, the two closed-form transfer functions $\theta_m \rightarrow \hat{\theta}$ and $w_{ym} \rightarrow \hat{\theta}$ - this is exactly the **complementary**

filter transfer-function pair $G_L(s)/G_H(s)$ documented in the lecture notes, just derived here from the KF gain rather than assumed a priori.

- 4.3: discuss effect of tuning Q (process noise weight) vs R (measurement noise weight) - same Bryson's-rule-style tradeoff as Ch4's LQR tuning discussion, applied here to the dual (estimation) problem.
- 4.4: **augment the state with a gyro-bias state** b_y , model $\dot{b}_y = 0$ and $w_{ym} = q + b_y$ (bias-augmented rate-gyro model). Suggested validation: inject an artificial bias in simulation and plot true vs. estimated bias to tune the filter. This is a **2-state, 2-output (or 2-state, single-channel with 2 measurement paths) special case** of exactly the same architecture as the full 6-state paper filter in Section 5, but restricted to a single axis (pitch, then roll) instead of jointly estimating a 3-vector attitude + 3-vector bias.
- 4.5: check whether the bias-free and bias-augmented filters are **complementary filters** in the formal sense ($G_L + G_H = 1$), and discuss the tradeoffs of that filter class (fast high-frequency response from the gyro, but drift if not bias-compensated; slow but bias-free low-frequency truth from the inclinometer).
- 4.6: relate results back to Experiments A-E (i.e. show how filter performance/steady-state error changes with the noise regime of each experiment).
- 4.7: repeat everything for **roll**, using the roll inclinometer and rate-gyro measurement w_{xm} (by symmetry with pitch/ w_{ym}).

Section 5 - Integrated roll and pitch estimation (the capstone task, "(L)" only, no separate theory sub-parts spelled out): implement the **Kalman filter for the "nominal system (2)"** of Batista/Silvestre/Oliveira (2011) - i.e. NOT the full nonlinear/GAS-error-dynamics observer from the paper's Theorem 3.4, but the simpler linear time-varying system (their Eq. 2) treated as a standard continuous-time Kalman filter (their Section 4, "Filter design"). The handout explicitly narrows scope: **only the accelerometer-based partial-attitude vector observation + 3-axis rate-gyro-bias estimation** is required, i.e. the exact 6-state filter of the paper ($x = [y; b_\omega] \in \mathbb{R}^6$), not a re-derivation of the observability proofs.

2 Reference paper - Partial attitude and rate gyro bias estimation (Batista, Silvestre, Oliveira 2011)

Problem: with only a **single** body-fixed vector observation of a known constant inertial vector (e.g. gravity via the accelerometer), full attitude ($R \in SO(3)$) cannot be recovered - one rotational degree of freedom (rotation about the observed vector, i.e. yaw when the vector is gravity) is fundamentally unobservable, since $y(t) = R^T(t)^I y = R^T(t) R_0^T I y$ for any rotation R_0 about $I y$. The paper's contribution: exploit **biased rate-gyro measurements** in addition to the single vector observation to (a) filter the vector observation itself, and (b) fully estimate the 3-axis gyro bias, with a filter that has **globally asymptotically stable (GAS)** error dynamics - stronger than typical EKF-based attitude filters, which only have local/linearization-based guarantees.

System model (their Eq. 1-2): let $y(t) \in \mathbb{R}^3$ be the vector observation in body coordinates ($I y = R(t)y(t)$, $I y$ constant, known, in the inertial frame),

$\omega_m(t) = \omega(t) + b_\omega(t)$ the biased rate-gyro reading, b_ω the (assumed constant) bias. Using $\dot{y} = -S(\omega)y$ (differentiating the observation equation, since $\dot{R} = RS(\omega)$) and substituting the biased measurement:

$$\begin{aligned}\dot{x}_1(t) &= -S(\omega_m(t))x_1(t) + S(x_2(t))x_1(t) & [x_1 = y, \text{ using } \omega = \omega_m - b_\omega \text{ and } x \times y = -y \times x] \\ \dot{x}_2(t) &= 0 & [x_2 = b_\omega, \text{ constant}] \\ y(t) &= x_1(t)\end{aligned}$$

Compact linear time-varying (LTV) form - **this is the crux of the whole paper**: although the system is *physically* nonlinear (the state x_2 multiplies x_1 in the dynamics), it can be rewritten so that the *output* $y(t)$ appears in the system matrix instead of the state, making it **linear time-varying for observer-design purposes** (the time-variation being driven by the measured/known signals $\omega_m(t)$ and $y(t)$, not by the unknown state):

$$\begin{aligned}\dot{x}(t) &= A(t)x(t), \quad y(t) = Cx(t), \quad x = [x_1; x_2] \in \mathbb{R}^6, \\ A(t) &= \begin{bmatrix} -S(\omega_m(t)) & -S(y(t)) \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{6 \times 6}, \quad C = [I, 0] \in \mathbb{R}^{3 \times 6}.\end{aligned}$$

This is the **"nominal system (2)"** the lab handout explicitly asks students to implement a Kalman filter for.

Observability analysis (Section 3, the paper's main theoretical contribution):

- Lemma 3.1: for a broader class of nonlinear systems expressible this way, observability of the underlying nonlinear system follows from invertibility of the **observability Gramian** of the associated LTV pair.
- Theorem 3.3: system (2) is observable on $[t_0, t_f]$ iff for every unit vector $d \in \mathbb{R}^3$, there exists $t^* \in [t_0, t_f]$ with $\|y(t^*) \times d\| > 0$ - i.e., **the vector observation $y(t)$ must not stay constant/parallel to any fixed direction throughout the interval** (the vehicle must actually rotate/maneuver relative to the observed vector). A vehicle sitting motionless in hover, with $y(t) = \text{constant}$ gravity direction in body frame, would make this system **unobservable** - directly analogous to, and a strict generalization of, the course's own "rate-gyro-only $\rightarrow$ not observable" worked example in [../lectures-22-23/notes/03-sensors-estimation.pdf](#) (there, a *constant-turn* roll estimator failed observability from yaw-rate alone; here, a *stationary* attitude/bias estimator fails observability from a single constant vector reading alone - both need an added, independent source of information/persistent excitation to become observable).
- Theorem 3.4 (stronger, needed for GAS filter design): **uniform complete observability** holds iff there exist $\alpha, \delta > 0$ such that for every unit d and every t , some $t^* \in [t, t + \delta]$ gives $\|y(t^*) \times d\| \geq \alpha$ - a **persistency-of-excitation** condition (not just "eventually observable," but observable with a uniform margin on every sliding window of length δ). This is the condition that actually gets invoked to prove the Kalman filter below has GAS error dynamics (via Kalman & Bucy 1961 / Jazwinski 1970 general stability results for uniformly completely observable LTV Kalman filters).

Filter design (Section 4): because (2) is treated as LTV (not nonlinear) for filtering purposes, the **standard continuous-time Kalman filter** applies directly,

with additive process/measurement noise $w(t)$, $n(t)$ (covariances Ξ , Θ):

$$\dot{\hat{x}}(t) = A(t)\hat{x}(t) + K(t)[y(t) - C\hat{x}(t)],$$

$$\dot{P}(t) = A(t)P(t) + P(t)A(t)^T + \Xi - P(t)C^T\Theta^{-1}CP(t) \quad (\text{Riccati differential equation}),$$

$$K(t) = P(t)C^T\Theta^{-1}.$$

Because $(A(t), C)$ is uniformly completely observable (Theorem 3.4), the filter error dynamics are **globally asymptotically stable** - this is the paper's headline result, and it is the reason the course picked this filter as the "advanced" capstone estimator for Lab 2 rather than a generic EKF.

Simulation results (Section 5): accelerometer (gravity vector) + biased rate gyro; injected bias $b_\omega = [2, -3, 1]^T \cdot \pi/180$ rad/s; sensor noise std 0.05 m/s² (accel) and 0.05°/s (gyro); $\Xi = \text{diag}(0.05I, 0.01I)$, $\Theta = 0.05I$. Recovers roll and pitch (the two Euler angles obtainable from a single vector = gravity) with small residual error (tenths of a degree) and converges the gyro-bias estimate within 20 s.

Experimental results (Section 6): validated against a precision motion rate table (Ideal Aerosmith 2103HT, 0.00025° resolution ground truth) with a low-cost MEMSENSE NANO IMU (accel std 0.008 m/s², gyro std 0.95°/s, 150 Hz) - i.e. sensor quality comparable to what the AR.Drone's onboard IMU is expected to have. Filter converges fast (< 1 s) even from a zero-bias initial guess, and steady-state roll/pitch error stays within about ± 0.5 -1° despite the raw accelerometer-only measurement being noisy ($\sim \pm 1$ -2°) - a concrete, quantified demonstration of the filtering benefit that Lab 2's Sections 3-4 ask students to reproduce qualitatively on the AR.Drone.

Important note for the lab report: the paper's $x_2 = b_\omega$ is the *full 3-axis* gyro bias and $x_1 = y$ is the (3-component) vector observation itself (not roll/pitch angles directly) - recovering roll and pitch from the filtered $\hat{x}_1$ requires the same $\text{atan2}/\sin^{-1}$ inversion documented in [../lectures-22-23/notes/03-sensors-estimation.pdf](#)'s "sensor-model inversion" section, applied post-filter instead of pre-filter (i.e. filter the raw vector first, then convert to angles - this generally gives better results than converting to angles first and filtering the angles, since the vector-space dynamics are the ones that are exactly LTV/exploitable, while angle-space dynamics involve trig nonlinearities).

3 Devkit guide/scripts - NAV-specific additions vs Lab 1's devkit

Script	Purpose	Notable content
start_here_NAV.m	Entry point, replaces Lab 1's start_here.m	Adds top-level option (3) Replay from stored data : prompts for a .mat filename, loads it, cd Replay, runs setupReplay. Options (1) Simulation and (2) Wi-Fi control are structurally identical to Lab 1's devkit (same sub-menus: baseline/hover/waypoint-tracking for simulation; hover/waypoint-tracking for Wi-Fi).

Script	Purpose	Notable content
lib/decode_32bit.m	Decodes raw 32-bit navdata fields from the AR.Drone's UDP packet stream	Takes a 4-byte input array, reassembles it (with byte-order swap: <code>hex_value(4,3,2,1)</code>), and returns either a <code>uint32</code> (mark=1), a signed value via <code>hex2dec(...)-2^31</code> (mark=2), or a single-precision float reinterpretation of the same 4 bytes (mark otherwise) via <code>typecast</code> . This is the low-level function that turns the drone's raw binary telemetry (battery status, accelerometer/gyro raw counts, etc.) into usable MATLAB numeric values - genuinely new vs. Lab 1, which per the handout only exposed higher-level velocity/attitude estimates, not raw IMU counts.
lib/getWaypoints.m	Same as Lab 1 - hardcoded 5-column waypoint list [$X_e, Y_e, h, \psi, \text{wait_time}$], edited per-mission (currently a small square path, take off $\rightarrow$ 2 corners $\rightarrow$ back $\rightarrow$ land, 5 s dwell each)	Unchanged in structure from Lab1; a large commented-out block shows an "ORIGINAL" 10-waypoint list, suggesting the file is meant to be edited per-experiment (e.g. for Experiment E's waypoint mission).
lib/setupARModel.m	Loads linearized transfer-function / state-space models (ssRoll, ssRoll2V, ssPitch, ssPitch2U, ssYaw, ssH) from .mat files	Identical to Lab 1's devkit; these are the small-signal linear models used by the <i>simulation</i> path only (not by the raw-navdata-based estimators built in this lab, which work directly on navdata).
Replay/setupReplay.m	Replay entry point	Just adds <code>../lib</code> to path and opens <code>ARDroneReplay_V2.slx</code> - the loaded .mat (containing a previously-recorded navdata variable, per <code>start_here_NAV.m</code> 's <code>load(filename)</code> call) is fed into this model instead of live Wi-Fi data, letting students re-run/develop estimators offline against Experiment A-E recordings without needing the physical drone each time.
simulation/setupBaseModel.m, setupHoverSim.m, setupWP-TrackingSim.m	Simulation-mode setup scripts	Unchanged from Lab 1's devkit: set <code>FMS.Ts=0.065s</code> , <code>timeDelay=4·FMS.Ts</code> , load the linear model, waypoints, <code>simDT=0.005s</code> , then open the corresponding .slx (<code>ARDroneBaseSim</code> , <code>ARDroneHoverSim</code> , <code>ARDroneWPTrackingSim</code>).

Script	Purpose	Notable content
wifiControl/ setupHover.m, setupWPTracking.m	Wi-Fi control setup scripts	Sample time now explicitly 1/200 (200 Hz, matching the new navdata logging rate quoted in the handout) rather than the simulation's 0.065s; both carry an explicit warning comment that the onboard position estimate is inaccurate because it integrates an already-inaccurate optical-flow-based velocity estimate - a direct, practical illustration of why Lab 2's whole point (building better estimators from raw IMU data) matters. Opens ARDroneHover_V2.slx / ARDroneWPTracking_V2.slx (the "_V2" suffix distinguishing these from Lab 1's plain ARDroneHover.slx/ARDroneWPTracking.slx, which are also still present in this devkit and are what Experiment D's instructions say to edit for the pitch-step test).

4 Simulink models present

All in DevKit_Lab2/ARDroneSimulinkDevKit_NAV/:

- lib/ARBlocks.slx - core AR.Drone block library (dynamics/interface blocks reused by every other model).
- simulation/ARDroneBaseSim.slx, ARDroneHoverSim.slx, ARDroneWPTrackingSim.slx - pure-simulation models (linear dynamics, no real drone).
- wifiControl/ARDroneHover.slx, ARDroneWPTracking.slx - original (Lab 1-era) Wi-Fi control models; per the handout, these are the ones Experiments D and E instruct students to modify (add a pitch step / a 2-waypoint y reference).
- wifiControl/ARDroneHover_V2.slx, ARDroneWPTracking_V2.slx - updated Wi-Fi control models with the extended navdata decoding (200 Hz raw accel/gyro logging) - these are the ones actually launched by setupHover.m/setupWPTracking.m and used to run Experiments A-C.
- Replay/ARDroneReplay_V2.slx - offline replay model consuming a previously-logged navdata variable instead of a live Wi-Fi feed.

5 Connections to lecture material

- **Raw pitch/roll from accelerometer (handout 3.4)** $\leftrightarrow$ the exact inclinometer formulas $\hat{\phi}_{accel} = \text{atan2}(y_{accel,y}, y_{accel,z})$ -style, $\hat{\theta}_{accel} = \sin^{-1}(y_{accel,x}/g)$ derived in [../lectures-22-23/notes/03-sensors-estimation.pdf](#) under "Sensor-model inversion," including the explicit caveat there that this approach fails during maneuvers (motivating everything that follows in both the lecture and the lab).

- **Steady-state KF for pitch (4.1-4.3)** $\leftrightarrow$ directly the LTI Kalman filter machinery from [../lectures-22-23/notes/02-control.pdf](#) (Ch4: `K=lqr(...)`-style tooling, duality with `L=place(...)`) and the continuous-discrete KF equations in `03-sensors-estimation.pdf` ($A_d = I + ATs + A^2Ts^2/2$, $P_{k+1} = A_dP_kA_d^T + Ts^2Q$), specialized here to MATLAB's built-in `kalman()/estim()` for the steady-state (algebraic Riccati) solution rather than a manually-coded recursive filter.
- **Bias-augmented KF (4.4)** $\leftrightarrow$ the exact "second-order complementary filter" structure documented in `03-sensors-estimation.pdf` ($x = [\phi, b_p]$, $u = p_m$, $y = \phi_m$, giving $\hat{\Phi}(s) = G_L(s)\Phi_m(s) + G_H(s)P_m(s)/s$) - the lab has students re-derive/re-tune this same structure for pitch and roll on real AR.Drone data.
- **Complementarity check (4.5)** $\leftrightarrow$ `03-sensors-estimation.pdf`'s explicit statement that "the complementary filter is a Kalman filter with fixed (non-adaptive) gain" - Section 4.5 of the lab is literally asking students to verify this equivalence ($G_L + G_H = 1$) for their own designed filters.
- **Section 5's paper-based filter** $\leftrightarrow$ generalizes the course's own three-stage worked example in `03-sensors-estimation.pdf` ("rate-gyro only $\rightarrow$ not observable," "+ accelerometer $\rightarrow$ observable"): the Batista/Silvestre/Oliveira filter is the natural full-3D, full-rigor version of that same idea - single vector observation (accelerometer/gravity) + biased rate gyro, formalized with a proper observability proof and a GAS convergence guarantee, instead of the course's simplified scalar/coordinated-turn special case.
- The **LQR $\leftrightarrow$ Kalman filter duality** explicitly taught in Ch4/Ch6 ($K = R^{-1}B^TP$ vs $L = \Sigma C^TN^{-1}$, dual Riccati equations) is exactly the mathematical machinery underlying the paper's Riccati equation $\dot{P} = AP + PA^T + \Xi - PC^T\Theta^{-1}CP$ and gain $K = PC^T\Theta^{-1}$ - same structure, just continuous-time LTV instead of LTI.
- The **AR.Drone-specific note in 03-sensors-estimation.pdf** that the AR.Drone has no GPS and instead uses sonar + downward-camera vision is corroborated here: `setupHover.m/setupWPTracking.m`'s explicit in-code warning that the onboard position estimate (integrated optical-flow velocity) is inaccurate is precisely why Lab 2 focuses only on attitude (roll/pitch) estimation from IMU data, not full position/velocity estimation.

6 Open questions / unclear points

- The handout's Eq. numbering ("system (2)") refers to the *paper's* equation 2, not a lab-internal numbering - confirmed by cross-checking against the paper itself (its Eq. (2) is exactly the compact LTV form $\dot{x} = A(t)x$, $y = Cx$ derived above).
- The handout does not specify numeric noise covariances (Q , R / Ξ , Θ) to use for the AR.Drone - students are expected to derive these from the Experiment A-C mean/covariance characterization in Section 3, unlike the reference paper which states its simulation/experimental covariances explicitly. Matching the AR.Drone's real IMU noise characteristics to the paper's simulated/experimental values (from a much higher-grade MEMSENSE IMU) is left entirely to the student.

- `decode_32bit.m`'s `mark=2` branch (`hex2dec(hex_value)-2^31`) is used for signed integers but the exact set of navdata fields that require each of the three mark modes (unsigned int / signed int / float) is not documented in the script itself - would need the Parrot AR.Drone SDK's navdata structure reference to map specific byte offsets to sensor channels.
- Which specific navdata fields (byte offsets into the NDS structure referenced in `decode_32bit.m`'s comment, e.g. accelerometer/gyro sample locations) are wired into the "extended version" navdata-decoding Simulink block is not visible from the .m files alone (the block internals are in the binary `ARBlocks.slx` / `ARDroneHover_V2.slx`, not inspected here) - would require opening these .slx files in Simulink to confirm exact scaling/units of the raw accelerometer and rate-gyro channels before Section 3's mean/covariance characterization can be done correctly.
- The paper's filter is explicitly noted (Remark 4) to not be truly "optimal" in a strict sense because of multiplicative noise (the $S(y(t))$ and $S(\omega_m(t))$ terms depend on noisy measured/estimated signals, not clean known signals) - the lab handout does not ask students to address this caveat, but it's worth flagging if a report reviewer probes filter optimality claims.